PYTHON FOR BEGINNERS · SESSION 7

Lists

Store many values in one variable: indexing, slicing, methods, list operations and loops over lists.

Recap

Ask the class:

- What does `break` do inside a loop?
- Which numbers does `range(2, 10, 3)` give?
- How does a `for` loop take items from a string?

What is a list?

A list is an **ordered collection of items** inside square brackets `[ ]`, separated by commas. One variable can hold **many values**.

lists.py

```
numbers = [10, 20, 30]
names = ["Aman", "Riya"]
mixed = [1, "two", 3.0, True]
empty = []
print(numbers, names, mixed, empty)
print(len(numbers))
print(type(numbers))
```

OUTPUT

```
[10, 20, 30] ['Aman', 'Riya'] [1, 'two', 3.0, True] []
3
<class 'list'>
```

- **Ordered:** every item has a position (index), starting from 0.
- **Mixed types allowed:** one list can hold numbers, text and more.
- **Mutable:** you can change, add and remove items after the list is made.
- **Duplicates allowed:** the same value can appear many times.

Making a list from other things

```
print(list("Python"))
print(list(range(1, 6)))
print("a b c".split())
```

OUTPUT

```
['P', 'y', 't', 'h', 'o', 'n']
[1, 2, 3, 4, 5]
['a', 'b', 'c']
```

Ask

What is the difference between `[1, 2, 3]` and `"123"`?

Indexing

Indexing works **exactly like strings**: the first item is at index **0**, and **negative** indexes count from the end.

Index positions

```
fruits = ["apple", "banana", "mango", "kiwi"]
           0        1         2        3
          -4       -3        -2       -1
```

```
fruits = ["apple", "banana", "mango", "kiwi"]
print(fruits[0])
print(fruits[2])
print(fruits[-1])
print(fruits[-2])
```

OUTPUT

```
apple
mango
kiwi
mango
```

Ask

What is `fruits[-4]`? What happens with `fruits[4]`?

Going past the end

Try it: read the error

```
fruits = ["apple", "banana", "mango", "kiwi"]
print(fruits[4])
```

ERROR

```
  File "your code", line 2, in <module>
IndexError: list index out of range
```

Lists can be changed

Unlike strings, a list is **mutable**. You can replace an item by assigning to its index.

```
fruits = ["apple", "banana", "mango"]
fruits[1] = "orange"
print(fruits)
```

OUTPUT

```
['apple', 'orange', 'mango']
```

Slicing

Syntax: `list[start:stop:step]`. The **stop index is not included**, and a slice **gives a new list**.

```
nums = [10, 20, 30, 40, 50, 60]
print(nums[1:4])
print(nums[:3])
print(nums[3:])
print(nums[::2])
print(nums[::-1])
print(nums[-2:])
```

OUTPUT

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 30, 50]
[60, 50, 40, 30, 20, 10]
[50, 60]
```

- Leaving out **start** means from the beginning, leaving out **stop** means to the end.
- A step of **-1** reverses the list.
- `nums[:]` makes a **copy** of the whole list.

Changing a slice

```
nums = [10, 20, 30, 40, 50]
nums[1:3] = [21, 31]
print(nums)
```

OUTPUT

```
[10, 21, 31, 40, 50]
```

List methods

Methods are called with a dot, like string methods. Unlike string methods, most list methods **change the list itself**.

Adding items

METHOD	WHAT IT DOES
`append(x)`	adds `x` at the **end**
`insert(i, x)`	adds `x` at **position i**
`extend(items)`	adds **every item** of another list at the end

```
fruits = ["apple", "banana"]
fruits.append("mango")
fruits.insert(1, "kiwi")
fruits.extend(["grapes", "orange"])
print(fruits)
```

OUTPUT

```
['apple', 'kiwi', 'banana', 'mango', 'grapes', 'orange']
```

Removing items

METHOD	WHAT IT DOES
`remove(x)`	removes the **first** `x`
`pop()`	removes and **returns** the last item
`pop(i)`	removes and returns the item at **position i**
`del list[i]`	removes the item at position `i`
`clear()`	removes **everything**

```
nums = [10, 20, 30, 20, 40]
nums.remove(20)
print(nums)
last = nums.pop()
print(last, nums)
first = nums.pop(0)
print(first, nums)
del nums[0]
print(nums)
nums.clear()
print(nums)
```

OUTPUT

```
[10, 30, 20, 40]
40 [10, 30, 20]
10 [30, 20]
[20]
[]
```

Searching

```
colors = ["red", "green", "blue", "green"]
print(colors.index("blue"))
print(colors.count("green"))
print("red" in colors)
print("pink" in colors)
```

OUTPUT

```
2
2
True
False
```

Sorting and reversing

```
nums = [5, 2, 9, 1]
nums.sort()
print(nums)
nums.sort(reverse=True)
print(nums)
nums.reverse()
print(nums)
print(sorted([3, 1, 2]))
```

OUTPUT

```
[1, 2, 5, 9]
[9, 5, 2, 1]
[1, 2, 5, 9]
[1, 2, 3]
```

`sort()` **changes the list**. `sorted()` leaves the list alone and **gives a new sorted list**.

Useful functions

```
marks = [72, 85, 60, 91]
print(len(marks), min(marks), max(marks), sum(marks))
print(sum(marks) / len(marks))
```

OUTPUT

```
4 60 91 308
77.0
```

These methods return None

Important

`append`, `sort`, `reverse` and similar methods change the list and **return None**. Do not write `nums = nums.sort()`.

```
nums = [3, 1, 2]
result = nums.sort()
print(result)
print(nums)
```

OUTPUT

```
None
[1, 2, 3]
```

List operations

```
a = [1, 2, 3]
b = [4, 5]
print(a + b)
print(b * 3)
print(2 in a)
print(7 not in a)
print(a == [1, 2, 3])
```

OUTPUT

```
[1, 2, 3, 4, 5]
[4, 5, 4, 5, 4, 5]
True
True
True
```

- `+` **joins** two lists into a new one.
- `*` **repeats** a list.
- `in` and `not in` check whether a value is **present**.
- `==` compares **all items in order**.

Copying a list

`b = a` does **not** copy the list. Both names point to the **same list**.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
c = a.copy()
c.append(5)
print(a)
print(c)
```

OUTPUT

```
[1, 2, 3, 4]
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
```

Ask

Why did `a` change when we only appended to `b`?

Loops over lists

Go through every item

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

OUTPUT

```
apple
banana
mango
```

When you also need the index

```
fruits = ["apple", "banana", "mango"]
for i in range(len(fruits)):
    print(i, fruits[i])
print("---")
for i, fruit in enumerate(fruits):
    print(i, fruit)
```

OUTPUT

```
0 apple
1 banana
2 mango
---
0 apple
1 banana
2 mango
```

`enumerate()` hands you the **index and the item together**, which is shorter and harder to get wrong.

With a while loop

```
fruits = ["apple", "banana", "mango"]
i = 0
while i < len(fruits):
    print(fruits[i])
    i += 1
```

OUTPUT

```
apple
banana
mango
```

Building a list in a loop

```
squares = []
for n in range(1, 6):
    squares.append(n * n)
print(squares)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

Start with an **empty list**, then `append()` inside the loop.

Picking items with a condition

```
nums = [4, 7, 10, 3, 8]
evens = []
for n in nums:
    if n % 2 == 0:
        evens.append(n)
print(evens)
```

OUTPUT

```
[4, 10, 8]
```

A shorter way (preview)

A **list comprehension** builds the same list in one line. You will study it later.

```
squares = [n * n for n in range(1, 6)]
print(squares)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

Lists inside lists

A list can hold **other lists**. This is how you store a **table of rows and columns**.

```
matrix = [[1, 2, 3], [4, 5, 6]]
print(matrix[0])
print(matrix[1][2])
for row in matrix:
    print(row)
```

OUTPUT

```
[1, 2, 3]
6
[1, 2, 3]
[4, 5, 6]
```

- `matrix[1]` is the **second row**.
- `matrix[1][2]` is the **third item of the second row**.
- A nested `for` loop visits **every item**.

```
matrix = [[1, 2, 3], [4, 5, 6]]
for row in matrix:
    for item in row:
        print(item, end=" ")
    print()
```

OUTPUT

```
1 2 3
4 5 6
```

Common mistakes

MISTAKE	WHAT HAPPENS	FIX
Index too big: `fruits[4]` in a 4-item list	IndexError	the last index is `len(list) - 1`
`nums = nums.append(5)`	nums becomes None	just write `nums.append(5)`
`remove()` of a missing item	ValueError	check with `in` first
`b = a` to copy	both names change together	use `a.copy()` or `a[:]`
Adding a list with `append`	a list appears inside the list	use `extend()` to add its items

A missing item

Try it: read the error, then check with in before removing

```
nums = [1, 2, 3]
nums.remove(10)
```

ERROR

```
  File "your code", line 2, in <module>
ValueError: list.remove(x): x not in list
```

append or extend?

```
a = [1, 2]
a.append([3, 4])
print(a)
b = [1, 2]
b.extend([3, 4])
print(b)
```

OUTPUT

```
[1, 2, [3, 4]]
[1, 2, 3, 4]
```

Practice questions

Basic questions

1. Make a list of 5 numbers and print the sum and the average.
2. Find the largest number in a list without using `max()`.
3. Count how many even numbers are in a list.
4. Print every item of a list with its index.
5. Print a list in reverse order without using `reverse()` or slicing.

Harder questions

6. Remove the duplicates from a list.
7. Find the second largest number in a list.
8. Join two lists and sort the result.
9. Take numbers from the user until they type 0, then print the list and its total.
10. Make a list of the squares of the numbers 1 to 10.

Find the output

Work out the output first, then press Run to check

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])
print(nums[::-1][0])
nums.append(60)
nums.pop(0)
print(nums)
print(len(nums))
```

Try the average question

Your turn: print the average of the marks, then press Run

```
marks = [72, 85, 60, 91, 78]
# print the sum and the average
```

Homework and wrap-up

- Write a program that takes **5 marks**, stores them in a list, and prints the **highest, lowest and average**.
- Write a program that **separates** a list of numbers into two lists: **even** and **odd**.
- Write a program that **rotates** a list by one position: `[1, 2, 3, 4]` becomes `[2, 3, 4, 1]`.
- Write a program that finds the **common items** of two lists.
- Revise the **list methods** tables, especially which methods **return None**.

Next class

Tuples and dictionaries.